

Análisis Funcional - Requisitos del negocio

1) Contexto

Antes de diseñar el modelo de datos es necesario identificar los elementos principales del juego y comprender cómo interactúan entre sí.

1. Qué entidades existen en el juego?
2. Cuáles son las propiedades de esas entidades?
3. Cómo se relacionan esas entidades entre sí?
4. Qué reglas y restricciones rigen el funcionamiento del juego?
5. Qué consultas o casos de uso debe cubrir el modelo?

Este análisis permite construir una base sólida para el diseño del modelo de entidad-relación.

2) Alcance

El juego se estructura en tres bloques funcionales principales:

A) Usuarios

El usuario es el protagonista del juego.

Se requiere:

- Registro e inicio de sesión.
- Gestión de perfil.
- Posibilidad de añadir o cambiar la imagen del avatar.
- Participar en partidas del juego.

La autenticación (email y contraseña) no se maneja dentro del dominio funcional del juego, ya que se delega a Supabase Auth, que gestiona credenciales de forma segura mediante hash criptográfico.

Los datos funcionales que se asocian a cada usuario incluyen:

- Un identificador único (UUID). Es la clave primaria. Coincide con el identificador generado por Supabase Auth.
- El nombre que se muestra.
- La ruta del avatar.
- El estado del usuario (activo o inactivo).
- La fecha de creación.

Se implementa una estrategia de borrado lógico, lo que significa que los usuarios no se eliminan de forma permanente, sino que simplemente se marcan como inactivos.

B) Juego

El juego consiste en partidas que se componen de preguntas.

Partida

Una partida es una sesión de juego que inicia un usuario.

Debe incluir:

- Un identificador único.
- El usuario que es propietario.
- La fecha
- La fecha de inicio y finalización.
- El número de errores cometidos.
- La puntuación final (premio).

Cada partida consta de 15 preguntas.

Preguntas

Cada pregunta debe contener:

- El texto del enunciado.
- El nivel de dificultad.
- La categoría.
- Una imagen asociada.

- Cuatro opciones posibles.
- Una única opción correcta representada mediante un índice numérico.

Respuestas

La respuesta es la selección que hace el usuario en una partida específica.

Debe incluir:

- Una referencia a la partida.
- Una referencia a la pregunta.
- La opción seleccionada.
- Un indicador de acierto o error.

C) Ranking

El ranking se refiere a la mejor puntuación histórica que ha logrado cada usuario.

Esto significa:

- Consultas agrupadas sobre las partidas.
- Cálculo de la puntuación más alta por usuario.
- Clasificación en orden descendente para mostrar la posición global.

3. Definiciones

3.1 Entidades

Las principales entidades que hemos identificado son:

- Usuario (profiles)
- Partida (game_sessions)
- Pregunta (questions)
- Categoría (categories)

3.2 Atributos

Usuario

- id
- display_name
- avatar_path
- is_active
- created_at

Partida

- id
- usuario_id
- puntuación
- total_preguntas
- created_at

Pregunta

- id
- texto
- categoria_id
- imagen_url
- opcion_a
- opcion_b
- opcion_c
- opcion_d
- correct_index

3.3 Relaciones

- Categoría (1) — (N) Pregunta
- Usuario (1) — (N) Partida
- Partida (1) — (N) Respuesta

- Pregunta (1) — (N) Respuesta

3.4 Reglas y restricciones

- Cada partida contiene exactamente 15 preguntas.
- No se repiten preguntas dentro de una misma partida.
- El ranking se basa en la mejor puntuación histórica del usuario.
- Cada pregunta tiene exactamente cuatro opciones (A-D). Sólo hay una respuesta correcta representada por `correct_index`.
- Cada pregunta tiene asociada una imagen.
- El usuario puede modificar su avatar.
- El usuario no se elimina físicamente, sino que puede desactivarse.

Estas reglas se implementarán mediante restricciones en el modelo relacional y validaciones en la lógica de la aplicación.

4. Tecnología

Para el desarrollo del juego, se plantea el uso de **Supabase** como plataforma de backend. Esta elección no se basa solo en la disponibilidad de una base de datos en la nube, sino porque reúne todo en un solo entorno:

- Base de datos relacional PostgreSQL.
- Sistema de autenticación y gestión de usuarios (Supabase Auth).
- Almacenamiento de archivos (Storage).
- API REST automática para acceder a los datos.

Esta integración nos permite contar con una arquitectura cliente-servidor completa, sin la necesidad de construir un backend desde cero.

4.1 Justificación de la elección

Se han considerado las siguientes necesidades del proyecto:

- Acceso para múltiples usuarios.
- Almacenamiento de datos en la nube.
- Manejo seguro de credenciales.

- Capacidad de escalar el sistema sin necesidad de rediseñar la arquitectura.
- Implementación de un modelo relacional que se alinee con los contenidos del módulo de bases de datos.

Dado que Supabase se basa en PostgreSQL, permite trabajar con un modelo relacional formal (tablas, claves primarias y foráneas, restricciones)

Además, proporciona una API REST automática (PostgREST), lo que facilita la comunicación entre la aplicación Android y la base de datos a través de solicitudes HTTPS que intercambian datos en formato JSON.

4.2 Gestión de autenticación mediante Supabase Auth

Un aspecto clave del entorno es cómo se gestionan los usuarios y las contraseñas.

En lugar de crear una tabla propia para almacenar las credenciales, se apuesta por utilizar **Supabase Auth**, que es el sistema de autenticación que viene integrado en la plataforma.

Esta elección se basa en varios principios importantes:

1. Separación de responsabilidades

La gestión de contraseñas es una tarea delicada que debe ser manejada por un sistema especializado.

2. Almacenamiento seguro de credenciales

Supabase Auth no guarda las contraseñas en texto plano.

Las contraseñas se convierten mediante un algoritmo de hash criptográfico (bcrypt) antes de ser almacenadas en la base de datos interna del sistema (auth.users).

Esto significa que ni el desarrollador ni la aplicación pueden acceder a la contraseña original.

3. Modelo basado en tokens

Después de iniciar sesión, Supabase genera un token de acceso (access_token).

Este token se envía con cada solicitud posterior a la API, lo que permite identificar al usuario autenticado sin necesidad de volver a enviar la contraseña.

4. Reducción del riesgo de seguridad

Al no gestionar las contraseñas manualmente:

- Se evita el almacenamiento inseguro.
- Se elimina la necesidad de implementar mecanismos de cifrado propios.
- Se reduce el riesgo de filtración de credenciales.